CYBERSECURITY MASTER ROADMAP // PHASE 1 FOUNDATIONS DEEP DIVE

DAY 01: CPU SILICON MECHANICS & x86_64 REGISTERS

Scope: Von Neumann Architecture • 5-Stage Execution Cycle • Register Slicing • Calling Conventions • Endianness • Stack Frames

[!] CYBERSECURITY RELEVANCE: WHY ARCHITECTURE MATTERS

Every binary exploit, buffer overflow, malware reverse engineering task, and low-level kernel protection bypass hinges directly on CPU silicon behavior and register states. If an attacker controls the Instruction Pointer (**RIP**), they dictate machine execution. If a defender understands Stack Frames and Register Calling Conventions, they can reconstruct intrusion timelines and write resilient memory mitigations.

01 // THE SILICON FOUNDATION & THE 5-STAGE EXECUTION CYCLE

1. FETCH	Control Unit reads the 64-bit memory address in **RIP**, fetching the instruction bytes from L1 instruction cache / RAM across the Address & Data Buses.
2. DECODE	Instruction Decoder translates raw opcodes (e.g., 0x48 0x89 0xE5) into micro-operations, determining source and destination registers.
3. EXECUTE	Arithmetic Logic Unit (ALU) performs arithmetic (ADD/SUB) or logical operations (AND/XOR/CMP), and evaluates branch conditions.
4. MEMORY ACCESS	If the instruction dereferences memory (e.g. mov rax, [rbp-8]), data is retrieved from or staged into L1/L2 data cache.
5. WRITE-BACK	Results are written into destination registers, status flags in **RFLAGS** are updated, and **RIP** advances to the next instruction.

02 // COMPLETE x86_64 REGISTER HIERARCHY & CALLING CONVENTIONS

64-BIT	32-BIT	16-BIT	8-BIT	PRIMARY TECHNICAL ROLE	CALLING CONVENTION (ABI)
RAX	EAX	AX	AL / AH	Accumulator / Math result	**Function Return Value & Syscall ID**
RBX	EBX	BX	BL / BH	Base Data Pointer	**Callee-saved General Register**
RCX	ECX	CX	CL / CH	Loop Counter / String Ops	**4th Function Argument**
RDX	EDX	DX	DL / DH	I/O & Arithmetic Data	**3rd Function Argument**
RSI	ESI	SI	SIL	Source String/Memory Pointer	**2nd Function Argument**
RDI	EDI	DI	DIL	Destination Memory Pointer	**1st Function Argument**
RBP	EBP	BP	BPL	Base / Frame Pointer	**Anchors current stack frame base**
RSP	ESP	SP	SPL	Stack Pointer	**Points to current stack top**
RIP	EIP	IP	-	Instruction Pointer	**Next instruction address (Exploit Target)**
R8	R8D	R8W	R8B	General Purpose	**5th Function Argument**
R9	R9D	R9W	R9B	General Purpose	**6th Function Argument**
R10-R15	R10D-R15D	R10W-R15W	R10B-R15B	Scratch / General computation	**R10/R11 caller-saved; R12-R15 callee-saved**
RFLAGS	EFLAGS	FLAGS	-	Condition & Status Register	**ZF (Zero), CF (Carry), SF (Sign), OF (Overflow)**

03 // PROCESS MEMORY SEGMENTS & ENDIANNESS MECHANICS

KERNEL SPACE (0x7FFFFFFFFFFF+)	Protected memory reserved strictly for OS Kernel Ring 0 drivers, system call tables, and hardware management. User-mode code generates a segmentation fault if direct access is attempted.
STACK SEGMENT (Grows Downward ↓)	High memory down to low memory. Contains function stack frames, local variables, function parameters, and Saved Return Addresses (RIP). Pointed to by **RSP**.
HEAP SEGMENT (Grows Upward ↑)	Low memory up to high memory. Houses dynamically allocated memory created at runtime via malloc(), calloc(), or new. Managed via brk() and mmap() syscalls.
BSS & DATA (Static Segments)	**BSS:** Uninitialized global and static variables (zeroed out by OS). **DATA:** Initialized global and static variables (e.g. int count = 10;).
TEXT (CODE) (Read-Only)	Compiled machine code instructions of the executable. Protected with non-writable permissions (Read + Execute) to prevent self-modifying code vulnerabilities.

[!] LITTLE ENDIAN VS BIG ENDIAN (MEMORY BYTE ORDERING)

Value: 0xDEADBEEFCAFE1337 (64-bit Quadword)

- **Little Endian (x86_64 CPU Default):** Least Significant Byte is stored at lowest address: \x37 \x13 \xFE \xCA \xEF \xBE \xAD \xDE
- **Big Endian (Network Byte Order):** Most Significant Byte is stored at lowest address: \xDE \xAD \xBE \xEF \xCA \xFE \x13 \x37

Exploit Rule: In buffer overflow exploits, all injected memory pointers must be byte-reversed into Little Endian format!

04 // THE FUNCTION CALL PROTOCOL & STACK FRAME ANATOMY

STACK FRAME LIFECYCLE & EXPLOITATION MECHANISM

- **1. The Call Instruction (call target):** CPU pushes the current Instruction Pointer (Saved RIP) onto the stack and jumps to target.

2. Function Prologue:

```
push rbp          ; Preserve caller's Base Pointer on stack
mov rbp, rsp      ; Anchor new frame base to current stack pointer
sub rsp, 0x20     ; Allocate 32 bytes for local function variables
```

3. Function Epilogue:

```
mov rsp, rbp      ; Collapse local variables
pop rbp           ; Restore caller's Base Pointer
ret               ; Pops Saved RIP from stack into RIP register (CPU resumes caller)
```

Buffer Overflow Vulnerability: If user input exceeds the allocated local variable buffer, data overwrites Saved RBP and then Saved RIP. When ret executes, the CPU jumps to the attacker's injected address.

PRACTICAL GDB VERIFICATION CHEATSHEET

- **Compile:** gcc -g -O0 registers_lab.c -o registers_lab (Debug symbols included, optimization disabled)
- **Launch:** gdb -q ./registers_lab (Silent start without legal banner)
- **Set Syntax:** set disassembly-flavor intel (Forces readable Intel assembly format)
- **Disassemble:** disassemble main (Views raw assembly instructions of main function)
- **Breakpoint:** break main (Halts execution at function entry point)
- **Inspect Registers:** info registers or i r (Prints all 16 GPR values in hex and decimal)
- **Examine Stack:** x/8gx $rsp (Examines 8 giant-words [8 bytes each] starting from stack pointer)